HPC-Practicals

1. Write a program to implement Parallel Bubble Sort and Merge sort using OpenMP.
Use existing algorithms and measure the performance of sequential and parallel algorithms.

- **Concept 1: Thread**

- A small execution unit (worker)
- Multiple threads run simultaneously to do work faster

- **Concept 2: Shared Memory**

shared(a, first)

- All threads use same array a
- No copy is created
- Changes by one thread are visible to others

- **Concept 3: Independent Work**

- Work is divided into non-overlapping parts
- (0,1) and (2,3) don't affect each other
- So safe to run in parallel

1. What is Bubble Sort?
* Simple meaning:
Bubble sort is a sorting algorithm where:
- We compare two nearby elements
- If they are in wrong order → swap them

* Example:

```
[5, 3, 2]

Step 1:
(5,3) → swap → [3,5,2]
(5,2) → swap → [3,2,5]

Step 2:
(3,2) → swap → [2,3,5]

Final:
[2,3,5]
```

- Biggest element "bubbles up" to the end
- That's why name = Bubble Sort

⚡ 2. What is Parallel Bubble Sort?
* Problem in normal bubble sort:
- Works one by one (sequential)
- Slow for large data

* Solution:
Use Parallel Bubble Sort (Odd-Even Sort)
We divide work like this:
Even phase:

```
(0,1), (2,3), (4,5)
```

Odd phase:

```
(1,2), (3,4), (5,6)
```

- These pairs are independent
- So they can run at same time (parallel)

🧵 3. What is OpenMP?
* OpenMP = Open Multi-Processing
- It is a library used in C/C++ for:
- Creating threads
- Running loops in parallel

* Key idea:
Instead of writing complex threading code:
You just write:

```
#pragma omp parallel for
```

- And compiler automatically:
- Creates threads
- Divides work

* Real-life example:
Teacher giving work:
Sequential:
- 1 student does all work
Parallel:
- 5 students → each does part

How the Code Works

Step 1: Create array

The program first creates an array in descending order:

100, 99, 98, ... , 1

This is the worst case for bubble sort because every element is in the wrong position.

Step 2: Run multiple passes
for (int i = 0; i < n; i++)

The loop runs n times.
Each pass helps move elements closer to their correct position.
After enough passes, the entire array becomes sorted.

Step 3: Decide phase (even or odd)
int first = i % 2;

This decides how comparisons will happen:

If i is even → start from index 0
Compare pairs like (0,1), (2,3), (4,5)
If i is odd → start from index 1
Compare pairs like (1,2), (3,4), (5,6)

This separation is important because these pairs do not overlap.

Step 4: Parallel comparison
#pragma omp parallel for

This line tells the system to use multiple threads.
Each thread works on different pairs at the same time.

So instead of checking one pair at a time, many comparisons happen together.

Step 5: Compare and swap
if (a[j] > a[j+1])

For each pair:

If the left element is greater than the right element
They are in the wrong order
So we swap them

This ensures smaller values move to the left and larger values move to the right.

Step 6: Repeat the process

The program repeats:

even phase
odd phase

again and again for n passes.

With each pass:

elements move closer to their correct position
gradually, the entire array becomes sorted

6. Final Result

At the end:

All elements are arranged in ascending order
The output becomes:

1 2 3 4 5 ... 100

The program then prints:

the sorted array
the time taken to perform sorting

- Merge Sort -

1. What is Merge Sort?

Simple meaning:
Merge Sort is a divide and conquer algorithm
- Divide the array into smaller parts
- Sort those parts
- Then merge them back in sorted order

Example:

```
[5, 3, 2, 8]

Step 1: Divide
[5,3]   [2,8]

Step 2: Divide again
[5] [3]   [2] [8]

Step 3: Merge (sorted)
[3,5]   [2,8]

Step 4: Final merge
[2,3,5,8]
```

Key idea:
- Break → Solve → Combine

2. What is Parallel Merge Sort?

Problem in normal merge sort:
- Left and right parts are solved one after another (sequential)

Solution:
Use Parallel Merge Sort
- Left part and right part are independent
- So we can process them at the same time

Concept:

```
Left half → Thread 1
Right half → Thread 2
```

After both finish → merge results

3. Use of OpenMP in Merge Sort
We use:

```
#pragma omp parallel sections
```

👉 This creates multiple sections that run in parallel

```
#pragma omp section
```

👉 Each section runs as a separate thread

4. How the Code Works -

Step 1: Create array
The program creates an array in descending order:

```
100, 99, 98, … , 1
```

This is a difficult case, so sorting work is more.

Step 2: Divide the array

```
int m = l + (r - l) / 2;
```

👉 Find middle index
👉 Split array into:
- Left part → (l to m)
- Right part → (m+1 to r)

Step 3: Parallel recursion

```
#pragma omp parallel sections
```

Inside this:

```
#pragma omp section
mergeSort(arr, l, m);

#pragma omp section
mergeSort(arr, m+1, r);
```

👉 Left half and right half are sorted simultaneously

Step 4: Continue dividing
This process repeats until:
- Each subarray has only one element
Example:

```
[5] [3] [2] [8]
```

Single elements are already sorted

Step 5: Merge step

```
merge(arr, l, m, r);
```

👉 Combine two sorted arrays into one sorted array
Example:

```
[3,5] + [2,8] → [2,3,5,8]
```

Step 6: Repeat merging
- Small arrays merge → bigger arrays
- Bigger arrays merge → final sorted array

5. Final Result
At the end:
- Entire array becomes sorted
- Output becomes:

```
1 2 3 4 5 … 100
```

The program prints:
- Sorted array
- Execution time

// Design and implement Parallel Breadth First Search and Depth First Search based on existing algorithms using OpenMP.
Use a Tree or an undirected graph for BFS and DFS.

1. What is BFS?
BFS means **Breadth First Search.**
It is a graph/tree traversal algorithm.
Simple meaning:
BFS visits nodes **level by level.**
Example tree:

```
      1
     / \
    2   3
   / \ / \
  4   5 6   7
```

BFS order:

```
1 2 3 4 5 6 7
```

Because it visits:

```
Level 0: 1
Level 1: 2, 3
Level 2: 4, 5, 6, 7
```

2. What is DFS?
DFS means **Depth First Search.**
Simple meaning:
DFS goes **deep first**, then backtracks.
Using same tree:

```
      1
     / \
    2   3
   / \ / \
  4   5 6   7
```

DFS order can be:

```
1 2 4 5 3 6 7
```

It first goes deep on one side, then comes back.

3. What is Parallel BFS?
In BFS, nodes at the same level can be processed together.
Example:

```
Level 1: 2 and 3
```

Both can be processed in parallel because they are independent.
OpenMP is used to create multiple threads for processing nodes at the same level.

4. What is Parallel DFS?
DFS is naturally harder to parallelize because it goes deep in one path.
But in OpenMP, we can parallelize checking adjacent nodes or child nodes.
Multiple threads can explore different branches.

BFS code runs like this
User enters data one by one.
insert() creates a binary tree level-wise.

Example input:

2, 4, 5, 89, 3, 3

Tree becomes:

```
     2
    / \
   4   5
  /\  /
 89  3 3
```

bfs(root) starts from root node.
Queue stores nodes level by level.
First: 2
Then: 4, 5
Then: 89, 3, 3
OpenMP processes nodes of the same level in parallel.
Output:
2 4 5 89 3 3

One-line explanation:

BFS creates a tree from user input and visits nodes level by level using a queue.

DFS code runs like this
User enters:
number of nodes, number of edges, starting node

Example:

4 4 1
User enters connected pairs:
1 2
1 3
1 4
2 4
Code creates an undirected graph using adjacency list.
1 → 2, 3, 4
2 → 1, 4
3 → 1
4 → 1, 2
DFS starts from starting node 1.
Stack is used to go deep first.
OpenMP checks adjacent nodes in parallel.
Possible output:
1 4 3 2

Order may change because threads run in parallel.

a) Implement Parallel Reduction using Min, Max, Sum and Average operations.

b) Write CUDA programs that, given an *N-element vector*, find Home Faculty of Engineering Savitribai Phule Pune University Syllabus for Fourth Year of Computer Engineering.

1. **The maximum element in the vector.**
2. **The minimum element in the vector.**
3. **The arithmetic mean of the vector.**
4. **The standard deviation of the values in the vector.**

Test for input N and generate a randomized vector V of length N (N should be large). The program should generate output as the two computed maximum values as well as the time taken to find each value.

a) Parallel Reduction Theory
What is Reduction?
Reduction means converting many values into one final value.
Example:

```
Array = [2, 5, 1, 8, 4]
```

Reduction operations:

```
Sum = 20
Min = 1
Max = 8
Average = 20 / 5 = 4
```

So, reduction is used to find:
- Minimum value
- Maximum value
- Sum
- Average

What is Parallel Reduction?
In normal/sequential reduction, one processor checks elements one by one.

```
2 + 5 + 1 + 8 + 4
```

In parallel reduction, the array is divided among multiple threads.
Example:

```
Array = [2, 5, 1, 8, 4, 6]
```

Thread 1:

```
2, 5
```

Thread 2:

```
1, 8
```

Thread 3:

```
4, 6
```

Each thread calculates a partial result.
Then all partial results are combined to get the final answer.

Why Parallel Reduction is Used?
Parallel reduction is used because:
- It reduces execution time
- It uses multiple threads
- It is useful for large arrays
- It improves performance

Min Reduction
Minimum reduction finds the smallest element.
Example:

```
Array = [10, 4, 8, 2, 15]
```

Output:

```
Minimum = 2
```

In parallel:
- Each thread finds local minimum
- Final minimum is calculated from all local minimum values

Max Reduction
Maximum reduction finds the largest element.
Example:

```
Array = [10, 4, 8, 2, 15]
```

Output:

```
Maximum = 15
```

In parallel:
- Each thread finds local maximum
- Final maximum is calculated from all local maximum values

Sum Reduction
Sum reduction adds all elements.
Example:

```
Array = [10, 4, 8, 2, 15]
```

Output:

```
Sum = 39
```

In parallel:
- Each thread calculates partial sum
- All partial sums are added to get final sum

Average Reduction
Average is calculated using sum.
Formula:

```
Average = Sum / Number of elements
```

Example:

```
Array = [10, 4, 8, 2, 15]

Sum = 39
Number of elements = 5

Average = 39 / 5 = 7.8
```

In parallel:
- First calculate sum using parallel reduction
- Then divide sum by total number of elements

OpenMP Reduction Clause
OpenMP provides a special reduction clause.
Syntax:

```
#pragma omp parallel for reduction(+:sum)
```

This means:
- Each thread gets its own private copy of sum
- Each thread calculates partial sum
- At the end, OpenMP combines all sums automatically
For min:

```
#pragma omp parallel for reduction(min:minVal)
```

For max:

```
#pragma omp parallel for reduction(max:maxVal)
```

b) CUDA Based Vector Operations
1. What is CUDA?
CUDA (Compute Unified Device Architecture) is a parallel computing platform developed by NVIDIA.
It allows programs to run on **GPU instead of CPU.**

2. Why CUDA is used?
- GPU has many cores (thousands)
- Can process large data in parallel
- Faster than CPU for vector operations

3. Problem Statement
Given a large vector of size **N**, find:
- Maximum element
- Minimum element
- Arithmetic mean
- Standard deviation

4. Key Concepts in CUDA
🔹 **Kernel**
- A function that runs on GPU
- Called from CPU

```
__global__ void functionName()
```

🔹 **Threads**
- Small workers inside GPU
- Each thread processes part of data

🔹 **Blocks**
- Group of threads

🔹 **Grid**
- Collection of blocks

🔹 **Execution Model**

```
CPU → launches kernel → GPU executes parallel threads
```

5. Steps to Solve Problem
Step 1: Input
- Take size **N**
- Generate or input vector

Step 2: Transfer Data
- Copy data from CPU to GPU memory

Step 3: Kernel Execution
Each thread processes one element:
- Compare for max/min
- Add for sum

Step 4: Reduction
Combine results:
- Final max
- Final min
- Final sum

Step 5: Mean Calculation
Formula:

```
Mean = Sum / N
```

Step 6: Standard Deviation
Steps:
1. Calculate mean
2. For each element:

```
(x - mean)^2
```

3. Sum all values
4. Divide by N
5. Take square root
Formula:

```
Std Dev = √( Σ(x - mean)² / N )
```

6. Performance Measurement
- Measure time using GPU timers or clock
- Compare CPU vs GPU

7. Advantages of CUDA
- High performance
- Faster execution for large datasets
- Efficient parallel computation

8. Limitations
- Requires NVIDIA GPU
- Complex programming compared to OpenMP

Q) Assignment : Vector and Matrix Operations Design parallel algorithm to Add two large vectors. Multiply Vector and Matrix. Multiply two N × N arrays using n2 processors.

Vector and Matrix Operations (Parallel Algorithm)

1. Vector Addition
Concept:

Two vectors are added element by element.

C[i] = A[i] + B[i]
Parallel Idea:
Each element is independent
So multiple threads can compute different elements at the same time
Example:
A = [1,2,3]
B = [4,5,6]

C = [5,7,9]

2. Vector-Matrix Multiplication
Concept:

A vector is multiplied with a matrix.

C[i] = Σ (A[j] * B[j][i])
Parallel Idea:
Each output element can be computed independently
Assign one thread per element
Example:
Vector: [1,2]
Matrix:
[3 4]
[5 6]

Result:
[13 16]

3. Matrix Multiplication (N × N)
Concept:

Multiply two matrices.

C[i][j] = Σ (A[i][k] * B[k][j])

Parallel Idea:
Each element of result matrix is independent
Use one thread per element

n² Processors Concept
For N × N matrix → N² elements
Use N² processors → one processor per element

Each processor computes one C[i][j]

Advantages of Parallel Approach
- Faster computation
- Efficient for large data
- Uses multiple cores